

개요

이 가이드는 BizPlay RAG 챗봇 API 위에 UI를 구축하는 **프론트엔드 개발자**를 위한 것입니다. UI에서 일반적으로 호출하는 모든 엔드포인트, 요청/응답 형식, 그리고 가장 일반적인 기능(채팅, 봇 관리, 문서 업로드, **텔레그램/카카오톡 채널 연동**)에 대한 종단 간(end-to-end) 흐름을 다룹니다.

- **Base URL:** `http://<base_url>/api/v1`
- **인증:** 모든 `/api/**` 엔드포인트는 공개되어 있습니다 — **Authorization** 헤더가 필요하지 않습니다.
- **Content type:** 문서 업로드(`multipart/form-data`)를 제외한 모든 본문은 `application/json`입니다.
- **명세:** OpenAPI / Swagger UI는 `http://<base_url>/swagger-ui/index.html`에서 확인할 수 있습니다.
- **새로 생성된 봇은 비활성화 상태로 시작합니다** — 운영자가 문서 업로드와 시스템 프롬프트 튜닝을 마친 뒤 `PATCH /api/v1/bots/{id}/enable`을 호출해 활성화합니다. 비활성화된 봇은 채팅에는 **409**를 반환하지만 채널 연동 설정은 허용하므로, 운영자는 활성화 전에 텔레그램/카카오톡을 미리 연결해 둘 수 있습니다.

응답 래퍼(envelope)

모든 JSON 응답은 다음과 같이 래핑됩니다:

```
{
  "success": true,
  "message": "선택적인 짧은 사람이 읽을 수 있는 메모",
  "data": { /* 엔드포인트별 페이로드, null일 수 있음 */ }
}
```

오류 시에는 `success: false`이며, `message`에 설명이 담기고, `data`는 null입니다:

```
{ "success": false, "message": "Bot is disabled", "data": null }
```

HTTP 상태 코드는 결과를 반영합니다 (**200** 정상, **400** 잘못된 요청, **404** 찾을 수 없음, **409** 충돌, **500** 서버 오류). UI는 먼저 HTTP 상태 코드로 분기한 다음, 사용자에게 표시할 문자열은 `message`에서 읽어야 합니다.

핵심 개념

개념	설명
Corporation (법인)	봇을 소유하는 테넌트입니다. <code>corp_no</code> (외부 로그인 서비스가 발급하는 사업자 식별 코드)로 식별합니다 — 소프트 참조(soft reference) 이므로 API는 값만 저장하며, 로컬에 일치하는 행이 있을 필요는 없습니다. 값이 지정되지 않을 때의 기본 테넌트 폴백을 위해 Default Corporation (<code>corp_no = "DEFAULT"</code>)이 시드되어 있습니다. 로컬의 <code>corp / corp_group</code> 행은 <code>/api/v1/corps</code> 와 <code>/api/v1/corp-groups</code> 로 관리합니다.
Bot (봇)	사용자가 상호작용하는 단위입니다. 모든 봇은 이름, 설명, 시스템 프롬프트, LLM 모델, 동작 설정(<code>temperature</code> , <code>history-turns</code> , <code>top-K</code> 등)을 가집니다. 문서와 채팅 세션은 정확히 하

개념	설명
	나의 봇에 속합니다.
Document (문서)	봇의 벡터 스토어로 수집된 업로드된 파일(PDF / DOCX / TXT)입니다. 문서의 청크는 채팅 중 검색되지만, 해당 문서의 봇과 대화할 때만 검색됩니다.
Chat session (채팅 세션)	멀티 턴 대화입니다. 세션은 봇 범위입니다: <code>sessionId</code> 는 해당 봇 내에서만 의미가 있습니다. 모델이 이전 턴을 볼 수 있도록 후속 채팅 호출에 동일한 <code>sessionId</code> 를 전달하세요. 첫 번째 메시지에서는 생략하세요 — 응답에 새 ID가 포함됩니다. 각 세션은 분석 대시보드에서 출처를 그룹핑할 수 있도록 <code>channel</code> 값(<code>"web"</code> / <code>"telegram"</code> / <code>"kakaotalk"</code>)을 가집니다.
Recommended question (추천 질문)	"다음 중에서 시도해 보세요..." 버튼을 표시하려는 채팅 UI를 위해 봇에 첨부된 짧은 프리셋 시작 프롬프트입니다. 텔레그램 채널에서는 <code>/start</code> 인사말 아래 인라인 키보드 버튼으로 노출됩니다.
Channel integration (채널 연동)	봇은 텔레그램 봇(long-polling, outbound HTTPS만 사용)과/또는 카카오톡 오픈빌더를 통한 카카오톡 채널(webhook + 비동기 callback)에 추가로 연결할 수 있습니다. 동일한 RAG 파이프라인이 모든 전송 방식을 처리하며, 세션은 그에 맞게 <code>channel</code> 값을 기록합니다. 아래 채널 연동 섹션을 참고하세요.

엔드포인트 레퍼런스

모든 URL은 base인 `/api/v1`에 상대적입니다.

봇 (Bots)

봇 생성

`POST /bots`

요청 본문 (`name`과 `llmModel`만 필수):

```
{
  "corpNo": "ACME-001",
  "name": "Travel Expense Bot",
  "description": "Helps employees with the travel reimbursement policy",
  "contactEmail": "travel-support@bizplay.com",
  "contactPhone": "+82-2-1234-5678",
  "systemPrompt": null,
  "sourceExpose": true,
  "llmModel": "exaone-3.5-7.8b",
  "llmTemperature": 0,
  "maxAnswerLength": 1024,
  "historyTurns": 5,
  "topK": 5,
  "recommendedQuestions": [
    { "question": "When can I claim travel expenses?" },
    { "question": "What is the per-diem rate for international trips?" }
  ]
}
```

```
    ]
  }
```

필드	타입	필수	기본값	설명
corpNo	String	아니요	"DEFAULT"	테넌트 — 사업자 식별 코드(최대 50자). 소프트 참조 — 법인 데이터는 외부 로그인 서비스가 소유하므로 어떤 값이든 허용되며 로컬 존재 여부는 검사하지 않습니다. 생략 시 설정된 기본 테넌트로 폴백됩니다.
name	String	Yes	—	최대 255자.
description	String	No	null	자유 형식.
contactEmail / contactPhone	String	No	null	순수 메타데이터.
systemPrompt	String	No	정적 기본값	생략하면 서버 측에서 합리적인 기본값이 적용됩니다. 봇의 목적에 맞는 프롬프트 초안을 작성하려면 먼저 POST /bots/generate-system-prompt 를 호출하세요.
sourceExpose	Boolean	No	true	false인 경우, 이 봇의 채팅 응답은 sources: [] 를 반환합니다.
llmModel	String	Yes	—	GET /rag/chat/models 의 이름과 일치해야 합니다.
llmTemperature	Number	No	0	범위 0.0-1.0.
maxAnswerLength	Integer	No	1024	LLM maxTokens . 범위 64-8192.
historyTurns	Integer	No	5	프롬프트에 전달되는 이전 턴 수. 범위 0-20.
topK	Integer	No	5	채팅당 검색되는 청크 수. 범위 1-50.
recommendedQuestions[]	Array	No	[]	인라인 시드.

응답: **BotResponse** (아래 **봇 상세 조회** 참조). **주의:** 요청 본문의 내용과 관계없이, 새로 생성된 봇은 항상 **disabled: true**로 반환됩니다. 운영자가 설정(문서, 프롬프트, 채널 연동)을 마친 뒤 **PATCH /bots/{id}/enable**을 호출해야 채팅을 받기 시작합니다.

봇 목록 조회

GET /bots → **corp_no**와 무관하게 시스템의 모든 봇(비활성화된 봇 포함)에 대한 **BotSummary[]**을 반환합니다. 특정 법인으로 필터링하려면 **GET /bots/by-corp/{corpNo}**를 사용하세요.

```
{
  "success": true,
  "data": [
```

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "corpNo": "ACME-001",
  "name": "Travel Expense Bot",
  "description": "Helps employees with the travel reimbursement policy",
  "llmModel": "exaone-3.5-7.8b",
  "disabled": false,
  "sourceExpose": true,
  "createdAt": "2026-04-29T10:00:00",
  "updatedAt": "2026-04-29T10:00:00"
}
]
```

`corpNo`와 `sourceExpose`는 요약(summary)에 포함되어 있어, 관리자 / 크로스 테넌트 UI에서 봇을 그룹핑하거나 필터링하고, 클라이언트 UI에서 출처(sources) 패널을 즉시 숨기는 작업을, 추가로 `GET /bots/{id}`를 호출하지 않고 처리할 수 있습니다.

법인(corp) 별 봇 목록 조회

`GET /bots/by-corp/{corpNo}` → `GET /bots`와 동일한 `BotSummary[]` 형식을 반환하지만, 단일 `corp_no`로 필터링됩니다. 비활성화된 봇도 포함됩니다. `corp_no`는 외부 로그인 서비스가 소유한 데이터에 대한 소프트 참조이므로, 알 수 없는 코드에는 404 대신 빈 목록이 반환됩니다.

관리자 / 크로스 테넌트 화면에서 전체 목록 대신 특정 법인의 봇만 표시해야 할 때 사용하세요.

봇 상세 조회

`GET /bots/{id}` → `recommendedQuestions[]`과 채널 연동 플래그를 포함한 전체 `BotResponse`:

```
{
  "success": true,
  "data": {
    "id": "550e8400-...",
    "corpNo": "ACME-001",
    "name": "Travel Expense Bot",
    "description": "...",
    "contactEmail": "...",
    "contactPhone": "...",
    "systemPrompt": "You are a helpful assistant ...",
    "sourceExpose": true,
    "llmModel": "exaone-3.5-7.8b",
    "llmTemperature": 0.0,
    "maxAnswerLength": 1024,
    "historyTurns": 5,
    "topK": 5,
    "disabled": false,
    "telegramConfigured": true,
    "telegramBotUsername": "BizPlayTravelBot",
    "telegramConfiguredAt": "2026-05-12T14:30:00",
    "kakaoConfigured": false,
  }
}
```

```

    "kakaoBotName": null,
    "kakaoConfiguredAt": null,
    "recommendedQuestions": [
      { "id": "uuid", "question": "When can I claim travel expenses?" }
    ],
    "createdAt": "2026-04-28T10:00:00",
    "updatedAt": "2026-04-28T10:00:00"
  }
}

```

필드	설명
<code>telegramConfigured</code>	봇이 텔레그램 봇에 연결되어 있으면 true. 토큰 자체는 절대 반환되지 않습니다.
<code>telegramBotUsername</code>	텔레그램 측 핸들(예: " <code>BizPlayBot</code> "). <code>t.me/{username}</code> 링크를 만들 때 유용합니다. 연결되지 않은 경우 null.
<code>kakaoConfigured</code>	봇에 Kakao i 오픈빌더 Skill 웹훅이 구성되어 있으면 true.
<code>kakaoBotName</code>	운영자가 지정한 표시용 라벨(예: " <code>출장규정봇</code> "). 카카오는 정규화된 이름을 제공하지 않습니다. 연결되지 않은 경우 null.
<code>kakaoConfiguredAt</code> / <code>telegramConfiguredAt</code>	연결이 생성된 시각. 연결되지 않은 경우 null.

봇별 시크릿이 포함된 카카오 웹훅 URL은 `BotResponse`에 **포함되지 않습니다**. 필요할 때 `GET /bots/{id}/kakao`로 별도 조회하세요(아래 채널 연동 섹션 참조).

봇 수정

`PUT /bots/{id}` — **PATCH 의미론**: 모든 필드는 선택 사항이며, 생략된 필드는 변경되지 않습니다. `recommendedQuestions`는 특별합니다:

<code>recommendedQuestions</code> 값	효과
<code>null</code> (또는 필드 생략)	기존 목록이 그대로 유지됩니다.
<code>[]</code> (빈 배열)	모든 추천 질문이 삭제됩니다.
<code>[{question:"..."}, ...]</code>	전체 목록을 원자적으로 교체합니다.

`corpNo`는 수정으로 변경할 수 없습니다.

비활성화 / 활성화

`PATCH /bots/{id}/disable` — 봇은 보존되지만 새 채팅은 **409 Conflict**를 반환합니다. 문서와 기록은 유지됩니다. `PATCH /bots/{id}/enable` — 위의 작업을 되돌립니다.

삭제

`DELETE /bots/{id}` — **하드 삭제**. 한 번의 트랜잭션(순수 JDBC)으로 봇의 벡터 청크, 디스크 파일, `bots` 행을 제거합니다. 이후 DB 레벨의 `ON DELETE CASCADE`가 `documents`, `chat_sessions`, `chat_messages`, `recommended`

questions를 정리합니다. 되돌릴 수 없습니다.

시스템 프롬프트 생성

[POST /bots/generate-system-prompt](#) — 상태 비저장(stateless) 유틸리티. 봇을 생성하지 **않습니다**.

```
{ "name": "Travel Expense Bot", "description": "Helps with travel reimbursement",
  "llmModel": "exaone-3.5-7.8b" }
```

→

```
{
  "success": true,
  "data": {
    "systemPrompt": "You are a helpful assistant specialising in the company's
travel reimbursement policy. ...",
    "llmModel": "exaone-3.5-7.8b"
  }
}
```

생성기는 설명의 언어를 따라갑니다(한국어 설명 → 한국어 프롬프트; 영어 → 영어). 봇 생성 폼의 "생성" 버튼으로 사용하세요. LLM 호출이 실패하면 정적 기본 프롬프트로 대체됩니다 — 클라이언트는 항상 사용 가능한 결과를 받습니다.

봇 통계

[GET /bots/{id}/statistics](#) → 단일 봇에 대한 집계 카운터를 반환합니다. 봇이 존재하지 않으면 **404**를 반환합니다.

```
{
  "success": true,
  "data": {
    "botId": "550e8400-...",
    "botName": "Travel Expense Bot",
    "documentCount": 12,
    "completedDocumentCount": 11,
    "processingDocumentCount": 1,
    "failedDocumentCount": 0,
    "chatSessionCount": 47,
    "messageCount": 286,
    "conversationTurnCount": 143,
    "recommendedQuestionCount": 4,
    "lastChatAt": "2026-04-28T15:42:11"
  }
}
```

필드	타입	설명
botId	UUID	편의를 위해 경로 파라미터를 그대로 반환합니다.
botName	String	GET /bots/{id}를 추가로 호출하지 않아도 되도록 비정규화되어 포함됩니다.
documentCount	Long	이 봇이 소유한 전체 문서 수 (임베딩 상태 무관).
completedDocumentCount	Long	임베딩 파이프라인이 성공적으로 완료된 문서 수.
processingDocumentCount	Long	현재 임베딩 중인 문서 수.
failedDocumentCount	Long	임베딩이 실패한 문서 수 (재업로드 필요).
chatSessionCount	Long	이 봇에 대해 시작된 고유 채팅 세션 수.
messageCount	Long	저장된 전체 채팅 메시지 수 (사용자 + 어시스턴트 합산).
conversationTurnCount	Long	사용자 역할(user) 메시지 수 — 사용자 프롬프트 1건 + 그에 대한 어시스턴트 응답을 한 턴(turn)으로 셉니다.
recommendedQuestionCount	Long	봇에 구성된 추천 시작 질문 수.
lastChatAt	DateTime	가장 최근 채팅 메시지의 타임스탬프. 봇이 한 번도 사용된 적이 없으면 null.

모든 카운터는 JPA count 쿼리(행 로딩 없음)로 산출되므로, 히스토리 크기와 무관하게 호출 비용이 낮습니다.

선택적 시간 윈도우(window)

?windowDays={N} 쿼리 파라미터를 추가하면 최근 N일 구간의 지표와 그 직전 같은 길이 구간의 지표가 함께 반환됩니다. 대시보드의 KPI 카드 ("이전 기간 대비 X% 증가") 표시에 사용하세요.

```
GET /bots/{id}/statistics?windowDays=7
```

추가되는 필드 (윈도우를 지정하지 않으면 모두 null):

필드	타입	설명
windowDays	Integer	요청한 윈도우 크기를 그대로 반환합니다.
windowStart	DateTime	윈도우 시작(포함) — 보통 windowEnd - windowDays.
windowEnd	DateTime	윈도우 끝(미포함) — 요청 시점의 현재 시각.
windowConversationCount	Long	윈도우 내에 시작된 세션 수.
previousWindowConversationCount	Long	직전 동일 길이 구간([windowStart - windowDays, windowStart))에 시작된 세션 수. 증감률은 클라이언트에서 계산하세요: (windowConversationCount -

필드	타입	설명
		$\text{previousWindowConversationCount}) / \text{previousWindowConversationCount}.$
<code>windowMessageCount</code>	Long	윈도우 내에 저장된 메시지 수.
<code>previousWindowMessageCount</code>	Long	직전 동일 길이 구간의 같은 지표.
<code>windowTokenUsage</code>	Long	윈도우 내 어시스턴트 메시지의 <code>input_tokens</code> + <code>output_tokens</code> 합계. LLM이 usage 블록을 반환하지 않는 경우(vLLM 일부 설정) 0. 대시보드의 "Token usage" KPI 카드에 사용됩니다.
<code>previousWindowTokenUsage</code>	Long	직전 동일 길이 구간의 같은 지표 — 토큰 사용량 카드의 증감률 계산용.
<code>channelDistribution</code>	<code>Map<String, Long></code>	윈도우 내 채널별 세션 수 (예: <code>web</code> , <code>telegram</code> , <code>kakaotalk</code>). 활동이 없으면 빈 맵. "Channel distribution" 파이차트용.
<code>languageDistribution</code>	<code>Map<String, Long></code>	윈도우 내 사용자 메시지의 감지된 언어별 카운트 (예: <code>ko</code> , <code>en</code>). 어시스턴트 응답을 중복 집계하지 않도록 사용자 역할 메시지로만 제한합니다. "Distribution of languages used" 위젯용.

봇의 일별 활동 시계열

`GET /bots/{id}/statistics/daily?windowDays=7` → 윈도우 내 모든 일자(0건 포함, **zero-fill**)에 대해 하루 한 개의 버킷을 시계열로 반환합니다. 활동이 없는 날도 `conversationCount: 0, messageCount: 0`으로 포함되므로 클라이언트가 빈 칸 채우기 없이 바로 차트로 그릴 수 있습니다. 봇이 존재하지 않으면 `404`, `windowDays`가 없거나 0 이하이면 `400`을 반환합니다.

```
{
  "success": true,
  "data": [
    { "date": "2026-04-23", "conversationCount": 5, "messageCount": 28 },
    { "date": "2026-04-24", "conversationCount": 3, "messageCount": 14 },
    { "date": "2026-04-25", "conversationCount": 0, "messageCount": 0 },
    { "date": "2026-04-26", "conversationCount": 9, "messageCount": 47 },
    { "date": "2026-04-27", "conversationCount": 12, "messageCount": 62 },
    { "date": "2026-04-28", "conversationCount": 15, "messageCount": 81 },
    { "date": "2026-04-29", "conversationCount": 18, "messageCount": 94 }
  ]
}
```

필드	타입	설명
<code>date</code>	LocalDate	서버 타임존 기준 일자. 차트 x축 레이블로 사용.
<code>conversationCount</code>	Long	<code>createdAt</code> 이 이 날짜에 속하는 세션 수.

필드	타입	설명
<code>messageCount</code>	Long	<code>createdAt</code> 이 이 날짜에 속하는 전체 메시지 수 (user + assistant).

서비스 레이어에서 두 개의 네이티브 집계 쿼리(`GROUP BY DATE(created_at)`)를 단일 실행으로 합치므로, 윈도우 길이와 무관하게 효율적입니다.

인기 키워드

`GET /bots/{id}/statistics/keywords?windowDays=7&limit=10` → 윈도우 [now - windowDays, now) 내 사용자 메시지에서 가장 자주 등장한 상위 `limit`개 키워드. 봇이 존재하지 않으면 `404`, `windowDays ≤ 0`이거나 `limit ∉ [1, 100]`이면 `400`. `limit` 기본값은 `10`입니다.

```
{
  "success": true,
  "data": [
    { "keyword": "출장", "count": 32 },
    { "keyword": "비용", "count": 28 },
    { "keyword": "항공권", "count": 21 },
    { "keyword": "숙박", "count": 18 },
    { "keyword": "rental", "count": 14 }
  ]
}
```

필드	타입	설명
<code>keyword</code>	String	1-3 단어로 구성된 명사구. 추출기는 사용자 메시지 코퍼스에 대해 LLM 토픽 추출을 수행하므로 "출장비", "항공권", "숙박비"와 같은 복합 명사구가 자연스럽게 등장합니다. 입력 메시지의 다수 언어를 따라 반환됩니다 — 한국어 대화는 한국어 키워드, 영어 대화는 영어 키워드.
<code>count</code>	Long	윈도우 내에서 해당 키워드(또는 동의어/활용형 — "비용을", "비용이", "비용은"은 모두 "비용"으로 합산)를 언급한 사용자 메시지의 추정 수.

추출 방식 — 요청마다 LLM 한 번 호출, 봇이 사용 중인 채팅 모델을 그대로 사용합니다. 시스템 프롬프트에서 "도메인 특화 토픽 명사구만" 추출하도록 지시하고, 일반 동사(`give`, `take`, `주다`, `받다`), 인사말(`please`, `감사합니다`), 대명사, 대화체 표현은 제외합니다. 모델이 형태소 단위로 이해하기 때문에 "business trip"이 두 개의 별개 토큰이 아닌 하나의 키워드로 묶입니다.

실패 처리 — 키워드 추출은 핵심 경로가 아니므로, LLM 호출이 타임아웃되거나 잘못된 JSON을 반환하거나 봇 모델에 접근할 수 없을 때 `500` 대신 빈 `data` 배열로 `200`을 반환합니다. 대시보드는 데이터 없음과 추출 실패를 구분 없이 동일하게 처리합니다.

비용 — 대시보드 로드당 채팅 한 회. 사용자 메시지 코퍼스는 약 30,000자(약 7K 토큰, EXAONE의 32K 컨텍스트 윈도우 안에서 안전한 한도)로 잘려 전달되며, 한도를 초과하면 가장 오래된 메시지부터 잘립니다. LLM 부하가 부담되면 서비스 메서드에 `@Cacheable`(TTL 5-15분)을 적용하거나, `@Scheduled` 잡으로 `bot_keyword_stats` 테이블에 사전 계산해 두는 방식을 고려하세요.

봇의 채팅 세션 목록 조회

GET /bots/{id}/sessions → 해당 봇에 속한 모든 세션 요약물 생성 시간 내림차순(최신순)으로 반환합니다. 봇이 존재하지 않으면 **404**, 봇은 존재하지만 세션이 없으면 빈 배열과 함께 **200**을 반환합니다. 관리자 / 대시보드 화면에서 대화를 나열할 때 사용하세요. 단일 세션의 전체 메시지 내용은 **GET /rag/chat/history/{sessionId}**로 조회합니다.

```
{
  "success": true,
  "data": [
    {
      "sessionId": "550e8400-e29b-41d4-a716-446655440000",
      "botId": "0580b2d3-ef97-4566-b095-c66b37c3257b",
      "createdAt": "2026-04-29T15:42:11",
      "lastMessageAt": "2026-04-29T15:48:33",
      "messageCount": 6
    }
  ]
}
```

필드	타입	설명
sessionId	UUID	GET /rag/chat/history/{sessionId} 로 전체 메시지를 조회할 때 사용됩니다.
botId	UUID	편의를 위해 경로 파라미터를 그대로 반환합니다.
createdAt	DateTime	세션 생성 시각 (보통 첫 사용자 메시지 시각과 동일).
lastMessageAt	DateTime	가장 최근 메시지의 타임스탬프. 세션에 메시지가 없으면 null .
messageCount	Long	이 세션에 저장된 전체 메시지 수 (사용자 + 어시스턴트 합산).

단일 집계(aggregate) JPQL 쿼리로 산출되므로, 세션 수가 많아도 N+1 문제 없이 동작합니다.

추천 질문 (개별 관리)

전체 **recommendedQuestions[]** 세트는 전체 update 페이로드를 보내지 않고 개별적으로 편집할 수도 있습니다:

HTTP 메서드	경로	본문
GET	/bots/{id}/recommended-questions	—
POST	/bots/{id}/recommended-questions	{ "question": "..."}
DELETE	/bots/{id}/recommended-questions/{questionId}	—

POST는 저장된 항목을 반환합니다(서버 할당 UUID 포함). 순서는 생성 시간 오름차순입니다.

테넌트 관리 (Tenant administration)

로컬의 **corp_group / corp** 행에 대한 CRUD입니다. 로컬 행은 디렉터리 역할과 기본 테넌트 폴백을 위해 유지되며, 봇은 **corp_no**를 소프트 참조로만 보관하므로 **corp** 삭제는 봇에 직접 영향을 주지 않습니다(봇의 **corpNo**가 단순히

dangling 상태로 남을 뿐, 의도된 동작입니다).

Corp groups

POST /corp-groups — 본문 { "corpGroupCd": "ACME-GRP" }. 응답:

```
{ "success": true, "data": { "id": 2, "corpGroupCd": "ACME-GRP" } }
```

HTTP 메서드	경로	설명
GET	/corp-groups	전체 목록을 corpGroupCd 순으로 반환합니다.
GET	/corp-groups/{id}	경로의 id는 corp_group_id (BIGSERIAL).
PUT	/corp-groups/{id}	PATCH 의미론. corpGroupCd만 갱신 가능. 충돌 시 409.
DELETE	/corp-groups/{id}	어떤 corp 행이라도 이 그룹을 참조하면 409. 먼저 해당 corp들을 이동/삭제하세요.

Corps

POST /corps — 본문 { "corpNo": "ACME-001", "corpGroupId": 2, "corpName": "ACME Holdings" }. 응답:

```
{
  "success": true,
  "data": {
    "id": 7,
    "corpNo": "ACME-001",
    "corpGroupId": 2,
    "corpName": "ACME Holdings",
    "createdDate": "2026-04-29T10:00:00"
  }
}
```

HTTP 메서드	경로	설명
GET	/corps	전체 목록을 corpNo 순으로 반환합니다. 선택 쿼리 파라미터 ?corpGroupId=N.
GET	/corps/{corpNo}	경로 식별자는 사업자 식별 코드입니다.
PUT	/corps/{corpNo}	PATCH 의미론. corpGroupId와 corpName만 갱신 가능. corpNo는 불변 — 변경하려면 삭제 후 재생성하세요.

HTTP 메서드	경로	설명
DELETE	/corps/{corpNo}	corp 행을 제거합니다. 이 corp_no를 참조하던 봇은 소프트 참조가 dangling 상태로 남습니다(의도된 동작).
오류 발생 상황		
400	검증 실패 (corpGroupCd 공백, corpName 누락 등)	
404	생성/수정 시 corp_group_id 또는 경로의 id/corpNo가 존재하지 않음	
409	corpGroupCd / corpNo 중복, 또는 비어 있지 않은 corp_group 삭제 시도	

문서 (Documents)

모델 목록

GET /rag/chat/models → 봇의 "LLM 모델" 드롭다운을 채우는 데 사용되는 배열:

```
{
  "success": true,
  "data": [
    { "name": "exaone-3.5-7.8b", "label": "EXAONE-3.5-7.8B", "isDefault": true },
    { "name": "gemma-4-8b", "label": "Gemma-4-8B", "isDefault": false }
  ]
}
```

봇 생성/수정 시 llmModel을 보낼 때 name 값을 사용하세요.

문서 업로드

POST /rag/documents/upload — 세 부분으로 구성된 multipart/form-data:

필드	타입	필수	설명
botId	UUID	Yes	문서가 속한 봇. 봇은 존재해야 하며 비활성화되지 않아야 합니다.
file	File	Yes	PDF, DOCX 또는 TXT. 최대 50MB.
title	String	Yes	채팅 출처에 사용되는 표시 제목.

응답 (즉시 반환됨; 임베딩은 아직 진행 중일 수 있음):

```
{
  "success": true,
  "data": {
    "id": "doc-uuid",
    "botId": "bot-uuid",
    "title": "Travel Policy 2026",
    "fileName": "travel_policy_2026.pdf",
  }
}
```

```

    "contentType": "application/pdf",
    "embeddingStatus": "PROCESSING",
    "createdAt": "2026-04-28T10:01:23"
  }
}

```

`embeddingStatus`가 `COMPLETED` (또는 `FAILED`)가 될 때까지 `GET /rag/documents/{docId}`를 폴링하세요.

문서 목록 (봇별)

`GET /rag/documents?botId={botId}` — 필수 쿼리 파라미터. 해당 봇의 문서를 최신순으로 반환합니다.

문서 다운로드

`GET /rag/documents/{docId}/download` — 원본 `Content-Type`과 함께 업로드된 파일을 인라인으로 반환합니다. 채팅 UI에서 "원본 보기" 링크의 `<a href>`로 적합합니다.

비 ASCII 파일명(한글, CJK, 라틴 악센트 등)은 RFC 5987로 처리됩니다. 응답 헤더는 `Content-Disposition: inline; filename*=UTF-8''<percent-encoded>` 형식으로 ASCII 폴백을 함께 포함합니다. 최신 브라우저는 UTF-8 형식을 자동으로 선택하며, 구버전 클라이언트는 ASCII 폴백을 사용합니다. 프론트엔드에서 별도 처리는 필요 없습니다 — `window.location.assign(documentUrl)` 또는 일반 `<a href>`를 사용하면 됩니다.

문서 삭제

`DELETE /rag/documents/{docId}` — 디스크의 파일 + 이 문서의 모든 벡터 청크를 제거합니다. 같은 봇의 다른 문서는 영향받지 않습니다.

채팅 (Chat)

메시지 보내기

`POST /rag/chat`

```

{
  "botId": "550e8400-e29b-41d4-a716-446655440000",
  "query": "What's the daily allowance for an international trip?",
  "sessionId": null
}

```

필드	타입	필수	설명
<code>botId</code>	UUID	Yes	질문할 봇. 비활성화된 경우 <code>409</code> 를 반환합니다.
<code>query</code>	String	Yes	사용자 질문.

필드	타입	필수	설명
<code>sessionId</code>	UUID	No	대화의 첫 번째 메시지에서는 생략하세요. 응답은 새로 생성된 <code>sessionId</code> 를 반환합니다 — 후속 호출에 다시 전달하세요. 다른 봇에 속한 <code>sessionId</code> 를 보내면 <code>400</code> 을 반환합니다.
<code>channel</code>	String	아니요	자유 형식 채널 태그 (최대 20자, 예: <code>"web"</code> , <code>"telegram"</code> , <code>"kakaotalk"</code> , <code>"slack"</code>). 세션 생성 시점에만 세션 행에 저장되며, 같은 세션의 후속 메시지에서는 무시됩니다 — 한 세션은 처음 열린 채널 값을 유지합니다. 생략 시 <code>"web"</code> 으로 기본 설정됩니다. 대시보드의 "Channel distribution" 파이차트에 사용됩니다.

응답:

```
{
  "success": true,
  "data": {
    "answer": "The daily allowance for international trips is ...",
    "sessionId": "session-uuid",
    "sources": [
      {
        "docId": "doc-uuid",
        "title": "Travel Policy 2026",
        "fileName": "travel_policy_2026.pdf",
        "snippet": "Article 5 (Per Diem) – international trips: $100/day...",
        "score": 0.87,
        "chunkIndex": 12,
        "documentUrl": "/api/v1/rag/documents/doc-uuid/download"
      }
    ]
  }
}
```

`sources`는 최대 1건으로 제한됩니다 — 관련도가 가장 높은 단일 청크만 노출합니다. 검색 파이프라인은 LLM의 컨텍스트 윈도우에 여전히 `topK` 청크를 공급하지만, UI에 표시되는 출처 목록은 가장 상위 결과 하나로 좁혀집니다. 인용 UX가 단순해지고 긴 목록이 채팅 말풍선을 차지하는 일이 사라집니다.

`sources`는 다음과 같은 경우 빈 배열입니다:

- LLM이 거부 문구 `"I don't have enough information to answer this question."`로 응답한 경우
- 또는 `bot.sourceExpose`가 `false`인 경우
- 또는 벡터 검색에서 청크가 하나도 반환되지 않은 경우(봇에 문서가 없거나, 쿼리와 관련된 문서가 없는 경우). 이 경우에도 답변은 여전히 LLM이 생성합니다 — 봇의 시스템 프롬프트 + "관련 문서가 없음" 지시어를 사용하므로, 운영자가 시스템 프롬프트에 정의한 사용자 정의 거부 문구(예: `"죄송합니다. 답변할 수 없는 질문입니다."`)와 언어 규칙이 사용자 언어로 자동 적용됩니다.

`score`는 리랭킹이 활성화된 경우 리랭커 점수입니다(0–1, 높을수록 좋음). 그렇지 않으면 `1 - cosineDistance`입니다. 출처 신뢰도를 색상으로 코드화하는 데 사용하세요.

대화 기록

GET /rag/chat/history/{sessionId} — 세션의 모든 메시지를 오래된 순으로 반환합니다.

```
{
  "success": true,
  "data": [
    { "role": "user", "content": "What's the daily allowance?", "createdAt": "..."},
    { "role": "assistant", "content": "The daily allowance is ...", "createdAt": "..."}
  ]
}
```

사용자가 페이지를 새로고침할 때 대화 상태를 복원하는 데 유용합니다.

채널 연동 (Channel integrations)

봇은 텔레그램 봇과/또는 카카오톡 채널에 연결할 수 있습니다. 두 채널 모두 동일한 RAG 파이프라인을 재사용하며, 차이점은 사용자 메시지와 봇 응답을 전달하는 전송 방식뿐입니다. 각 봇별로 독립적으로 설정하며, 여러 봇을 동시에 연결할 수도 있고, 한 봇이 텔레그램만, 카카오톡만, 둘 다, 또는 어느 것도 가지지 않을 수 있습니다.

채널	공개 인그레스 필요 여부	라이프사이클	다중 인스턴스 안전
텔레그램	불필요 — api.telegram.org 로의 outbound long polling	연결된 봇당 백엔드 데몬 스레드 1개	불가능(텔레그램은 같은 토큰에 대한 두 번째 polling을 409로 거부)
카카오톡	필요, /api/v1/kakao/webhook/** 경로에 한정 — 나머지 /api/** 는 내부망에 유지	무상태 POST + 비동기 callback	가능

비활성화된 봇도 연결 가능합니다 — 채널 메시지에는 봇이 활성화되기 전까지 같은 채널을 통해 정중한 "현재 사용할 수 없음" 응답이 돌아갑니다. 따라서 운영자 흐름은: 봇 생성 → 문서 업로드 → 프롬프트 튜닝 → 텔레그램 / 카카오톡 연결 → 활성화.

텔레그램 (Telegram)

토큰은 텔레그램의 [@BotFather](#)가 발급합니다. 프론트엔드는 토큰을 받아 우리 API로 POST 하면 그 이후의 작업 (polling 스레드, 메시지 라우팅, 응답, 사용자별 세션 연속성)은 모두 서버 측에서 처리됩니다.

POST /bots/{id}/telegram — 텔레그램 봇 연결 또는 교체.

```
{ "token": "1234567890:ABCdefGHIjklMNOpqrSTUvwxyz-1234567890" }
```

응답(토큰은 절대 다시 반환되지 않음):

```
{
  "success": true,
}
```

```

"data": {
  "telegramConfigured": true,
  "botUsername": "BizPlayTravelBot",
  "configuredAt": "2026-05-12T14:30:00"
}
}

```

오류: 400 잘못된 토큰(텔레그램의 `getMe`가 거부함), 404 봇 없음, 409 토큰이 이미 시스템 내 다른 봇에 연결됨.

GET `/bots/{id}/telegram/status` — { `telegramConfigured`, `botUsername`, `configuredAt` }. 토큰은 반환되지 않습니다.

DELETE `/bots/{id}/telegram` — 연결 해제. polling 스레드를 정지하고, 텔레그램 측 잔여 웹훅을 best-effort로 제거하며, 행을 초기화합니다. { `telegramConfigured: false` } 반환.

운영 노트:

- 연동 방식은 **long polling** — 연결된 봇마다 우리 백엔드가 `api.telegram.org`로 long-lived HTTP 호출을 엽니다. 운영자가 노출할 웹훅 URL이 필요 없습니다.
- 인스턴스 1개만 운영 가능.** 텔레그램은 같은 토큰에 대해 두 번째 `getUpdates`가 들어오면 409로 응답합니다. 다중 복제 배포가 필요하면 리더 선출이 선행되어야 합니다.
- 1대1 비공개 채팅 전용.** 그룹/슈퍼그룹/채널 채팅은 "비공개 채팅 전용" 안내 한 줄로 응답합니다.
- 테스트 UI의 텔레그램 연결/해제 패널은 `qrcode-generator` 라이브러리로 `t.me/{username}` URL의 **스캔 가능한 QR 코드**도 렌더링합니다. 운영자가 핸드폰에서 사용자에게 봇 URL을 바로 전달할 수 있습니다.
- 봇 응답은 인용(`reply_to_message_id` 없이) 없는 독립 메시지로 전송되며, 4000자 줄바꿈 경계에서 분할됩니다 (텔레그램의 메시지당 최대 4096자 제한). LLM이 출력하는 마크다운은 텔레그램용 HTML로 변환됩니다.

카카오톡 (Kakao i 오픈빌더)

카카오는 인바운드 웹훅만 지원합니다. 운영자는 **Kakao i 오픈빌더**에 Skill을 만들고 Skill 설정에 **우리 웹훅 URL**을 붙여 넣습니다. 그러면 메시지는 카카오톡 → 우리 `/api/v1/kakao/webhook/{botId}/{secret}` → RAG 파이프라인 → 카카오톡으로 비동기 callback 순으로 흐릅니다.

POST `/bots/{id}/kakao` — 연결 또는 **웹훅 URL 회전**. 다시 호출하면 새 시크릿이 생성되고, 이전 URL은 즉시 동작을 멈춥니다.

```

{ "botName": "출장규정봇" }

```

응답에는 운영자가 오픈빌더에 붙여넣을 전체 웹훅 URL이 포함됩니다(경로에 봇별 시크릿이 들어 있으므로 민감 정보로 취급):

```

{
  "success": true,
  "data": {
    "kakaoConfigured": true,
    "botName": "출장규정봇",
    "webhookUrl":
      "https://chat.example.com/api/v1/kakao/webhook/550e8400-.../9f7b1c8d4e6f0a2b1c8d4e

```



```
6f0a2b1c8d",
  "publicBaseUrlMissing": false,
  "configuredAt": "2026-05-12T14:30:00"
}
```

필드	설명
<code>webhookUrl</code>	봇별 시크릿이 포함된 전체 URL. 인증 정보로 취급하세요 — 이 URL을 가진 누구나 봇에 메시지를 주입할 수 있습니다.
<code>publicBaseUrlMissing</code>	서버 측 <code>KAKAO_PUBLIC_BASE_URL</code> 환경 변수가 설정되지 않았으면 <code>true</code> 이며, 이 경우 <code>webhookUrl</code> 은 null. UI는 백엔드에 환경 변수 설정을 요청하라는 경고를 표시해야 합니다.

GET /bots/{id}/kakao — 동일한 형식으로 현재 상태를 조회(운영자가 봇 편집기를 열 때 URL을 지연 로드하기 위해 사용하세요. 봇 목록 페이로드에 URL이 노출되지 않도록 합니다).

DELETE /bots/{id}/kakao — 연결 해제. 시크릿 + 표시명 + 타임스탬프를 제거합니다. 이전에 발급한 웹훅 URL은 즉시 **404**를 반환합니다.

운영 노트:

- **공개 인그레스 필요** — `/api/v1/kakao/webhook/**` 경로에만 한정. 에지 프록시(nginx / Cloudfront 등)가 이 경로 접두사만 내부 앱으로 전달하도록 설정하고, 나머지 `/api/**`는 내부망에 유지하세요.
- **5초 동기 ACK + 비동기 callback.** 카카오는 5초 내 응답을 기대하지만 LLM 호출은 10–30초가 걸립니다. 서버는 `useCallback: true` + 임시 응답 말풍선("잠시만 기다려 주세요. 답변을 준비하고 있습니다...")을 동기 로 반환한 뒤, LLM이 끝나면 `userRequest.callbackUrl`로 실제 답변을 POST합니다. 표준 오픈빌더 패턴입니다.
- **콜백 기능은 카카오 비즈 승인이 필요합니다.** 봇의 오픈빌더 계정에서 **콜백 사용**이 승인되기 전까지는 실제 채팅 트래픽에도 `callbackUrl`이 함께 오지 않으며, 우리 앱은 LLM을 실행하지 않고 "✓ 웹훅 도달 가능" 메시지로 응답합니다. 오픈빌더의 **콜백 사용 신청** 양식으로 신청하세요.
- **다중 인스턴스 안전.** 웹훅은 무상태 POST입니다; 로드 밸런서 뒤의 여러 앱 인스턴스가 모두 동작합니다. 톤당 상태는 카카오가 주는 `callbackUrl`뿐이며, 한 번 사용하고 잊습니다.
- **메시지 렌더링:** 답변은 최대 3개의 `simpleText` 말풍선(각 1000자, 카카오의 `template.outputs × simpleText.text` 제한)으로 단락 / 줄 / 공백 경계에서 분할됩니다.

프론트엔드 설정 흐름(테스트 UI는 이 흐름 전부를 구현합니다. 커스텀 UI도 동일한 구조를 따릅니다):

1. POST `/api/v1/bots/{id}/kakao` body { `botName: "..."` }
→ 응답으로 `webhookUrl` 수신.
2. `webhookUrl`을 등쪽 코드 블록과 "복사" 버튼으로 표시.
`response.publicBaseUrlMissing === true` 인 경우 경고를 표시.
3. 운영자가 URL을 오픈빌더 `Skill` 설정에 붙여넣고,
"콜백 사용"을 켜고, 저장 후 봇을 배포.
4. 운영자가 실제 카카오톡 메시지를 보내면 사용자는

2초 이내 placeholder 말풍선을 보고, 10-30초 뒤 실제 답변을 받습니다.

5. (필요 시) URL 회전을 위해 다시 POST, 또는 완전히 해제하려면 DELETE.

웹훅 엔드포인트 자체(`POST /api/v1/kakao/webhook/{botId}/{secret}`)는 카카오가 호출하는 용도입니다 — 운영자나 프론트엔드가 직접 호출하지 않습니다. 경로 시크릿 인증: 시크릿이 잘못되었거나 누락되면 `404`("봇 없음"과 동일한 응답으로 통합해 공격자가 유효 botId를 탐색하지 못하도록 함)를 반환합니다.

종단 간(End-to-end) 레시피

A. 단일 봇용 채팅 UI 구축

1. GET `/api/v1/bots` → 봇 선택, id를 로컬에 저장
2. GET `/api/v1/bots/{id}/recommended-questions` → 빠른 선택 칩으로 렌더링
3. POST `/api/v1/rag/chat`
`{ botId, query, sessionId: null }` → 첫 번째 메시지;
`response.sessionId` 기억
4. POST `/api/v1/rag/chat`
`{ botId, query, sessionId }` → 후속 메시지, 동일한 `sessionId`

참고:

- `data.answer`는 마크다운 렌더링으로 표시하세요(모델은 마크다운을 출력합니다).
- `data.sources`는 접을 수 있는 카드로 표시되어야 하며, 기존의 "I don't have enough information..." 검사를 따르세요(해당 경우 빈 `sources` 배열은 의도적입니다).

B. 봇 생성 폼

1. GET `/api/v1/rag/chat/models` → LLM 드롭다운 채우기
2. POST `/api/v1/bots/generate-system-prompt`
`{ name, description, llmModel }` → 선택적 "생성" 버튼
3. POST `/api/v1/bots`
`{ name, description, llmModel, systemPrompt, ..., recommendedQuestions[] }`

생성 후, 응답에는 새 봇의 전체 구성이 포함됩니다(시드된 추천 질문과 서버 측 UUID 포함). 두 번째 GET은 필요하지 않습니다.

C. 문서 관리

1. POST `/api/v1/rag/documents/upload` (multipart: botId + file + title) → docId
2. Poll GET `/api/v1/rag/documents/{docId}` 를 약 3초 간격으로 `embeddingStatus =`

COMPLETED 가 될 때까지

3. List GET /api/v1/rag/documents?botId={botId}
4. Delete DELETE /api/v1/rag/documents/{docId}

봇의 벡터 체크는 `embeddingStatus = COMPLETED`인 순간 검색에 사용할 수 있습니다. "게시(publish)" 단계는 없습니다.

D. 대화 복원

1. 클라이언트 저장소(예: localStorage)에서 sessionId 읽기
2. GET /api/v1/rag/chat/history/{sessionId} → 메시지 렌더링
3. 동일한 sessionId로 POST /api/v1/rag/chat 계속 호출

사용자가 봇을 전환하면 저장된 sessionId를 버리세요 — 다른 botId와 함께 보내면 400을 반환합니다.

오류 처리

HTTP	발생 상황	일반적인 UI 동작
200	성공	data 렌더링
400	검증 실패 (예: 빈 query, 잘못된 temperature, sessionId가 다른 봇 소속, JSON 파싱 오류)	해당 입력 근처에 message 표시
404	봇, 문서 또는 세션을 찾을 수 없음	토스트 / 리다이렉트
409	봇이 비활성화됨 또는 채팅 호출이 차단된 상태에 도달	토스트: "이 봇은 비활성화되었습니다"
500	서버 측 버그	일반적인 "문제가 발생했습니다" 토스트; 백엔드팀에 message 보고

팁: 새 클라이언트의 일반적인 400은 `**JSON parse error: Illegal unquoted character (CTRL-CHAR, code 10)**`입니다 — 여러 줄 문자열(예: systemPrompt)은 JSON 값 내에서 줄바꿈이 \n으로 이스케이프되어야 합니다.

주목할 만한 사항

1. 세션은 봇 범위입니다. 봇 간에 sessionId를 재사용하지 마세요 — 사용자가 전환하면 새 대화를 시작하세요.
2. POST /rag/chat은 model / topK / exposeSources를 받지 않습니다. 이러한 설정은 봇에 있습니다. 동작을 변경하려면 PUT /bots/{id}를 통해 봇을 업데이트하세요.
3. 새로 생성된 봇은 비활성화 상태입니다. POST /bots는 요청 본문 내용과 관계없이 새 봇을 disabled: true로 반환합니다. 운영자가 문서 업로드와 프롬프트 튜닝을 마친 뒤 PATCH /bots/{id}/enable을 호출합니다. 텔레그램과 카카오톡 연동은 봇이 비활성화된 상태에서도 가능합니다.

4. **문서는 업로드 후 폴링이 필요합니다.** `embeddingStatus`는 임베딩 파이프라인이 완료된 후에만 `PROCESSING` → `COMPLETED` (또는 `FAILED`)로 전환됩니다. 큰 파일(>5MB)은 몇 분이 걸릴 수 있습니다. 한글 등 비 ASCII 파일명은 RFC 5987 인코딩으로 다운로드 엔드포인트에서 정상 전달됩니다.
5. **봇 삭제는 강력하게 연쇄됩니다.** `DELETE /bots/{id}`를 호출하기 전에 명시적으로 확인하세요 — 벡터 청크, 디스크의 파일, 채팅 기록, 그리고 텔레그램/카카오 채널 연결까지 모두 사라집니다. 봇이 텔레그램에 연결되어 있었다면 삭제 시 텔레그램 측 `deleteWebhook`도 best-effort로 호출됩니다.
6. **출처 목록은 1건으로 제한됩니다.** 봇의 `topK` 값과 무관하게 `data.sources`는 최대 1건만 반환됩니다. 이 제한은 무조건 적용되며, `topK`는 LLM이 내부적으로 보는 청크 수만 결정합니다.
7. **거부 문구는 봇의 시스템 프롬프트에 따릅니다.** 기본 시스템 프롬프트를 사용하는 봇은 LLM이 답할 수 없을 때 정확한 영어 문자열 `"I don't have enough information to answer this question."`을 반환합니다(이 문자열로 일치 검사를 해서 사용자 정의 거부 UI를 렌더링할 수 있습니다). 자체 거부 문구를 정의한 사용자 정의 시스템 프롬프트(예: `"답변이 불가능한 질문에 대해서는 '죄송합니다. 답변할 수 없는 질문입니다.'라고 답변해주세요."`)를 사용하는 봇은 그 문자열을 사용자 언어로 반환합니다. 이는 컨텍스트에서 답을 만들 수 없을 때뿐 아니라 문서가 전혀 없는 경우에도 동일하게 적용됩니다.
8. **기본 LLM 동작은 결정적(deterministic)입니다.** `llmTemperature = 0`은 같은 질문이 같은 답을 반환한다는 의미입니다. UX가 너무 경직되어 보이면 봇 생성/수정 시 `temperature`를 높이세요(최대 `1.0`).
9. **출처 URL은 상대 경로입니다.** `documentUrl`은 `"/api/v1/rag/documents/{id}/download"`을 반환합니다 — 하이퍼링크하려면 API base URL을 접두사로 붙이세요.
10. **채널 연동 토큰은 쓰기 전용입니다.** `BotResponse`는 `telegramConfigured` / `kakaoConfigured` 플래그와 비공개가 아닌 표시명을 담지만, 텔레그램 봇 토큰이나 카카오 웹훅 URL(봇별 시크릿이 경로에 포함됨)은 절대 반환하지 않습니다. 카카오 웹훅 URL을 초기 구성 호출 이후에도 다시 얻으려면 `GET /bots/{id}/kakao`를 호출하세요.